

Adaptive Quantization for Stable Knowledge Acquisition in Quantization-Aware Continual Learning

De Cheng, Kun Gu, Lingfeng He, Huaijie Wang, Di Xu, Nannan Wang, *Senior Member, IEEE*,
Xinbo Gao, *Fellow, IEEE*

Abstract—With the growing demand for efficient and adaptive intelligence on edge devices, deploying deep models under limited memory and computation constraints has become increasingly important. However, existing quantization-aware training (QAT) methods, primarily designed for single-task learning, struggle in dynamic environments where models must learn from a continuous stream of tasks. This often leads to severe catastrophic forgetting and significant representation drift. While continual learning (CL) offers mechanisms for preserving prior knowledge, it typically incurs substantial computational overhead. To address these challenges, we propose Adaptive Quantization for Stable Knowledge Acquisition (AQCL), a unified framework that integrates QAT and CL to achieve both hardware efficiency and lifelong adaptability. AQCL preserves essential knowledge while maintaining model plasticity under low-bit quantization. The framework incorporates two key components: Representation-Preserving Quantization (RPQ), which combines sensitivity-aware bit allocation with hierarchical feature fusion to preserve layers that encode critical information; and Similarity-Aware Orthogonal Update (SAOU), which mitigates feature drift by retaining gradient components orthogonal to prior task subspaces while adaptively scaling task-relevant directions to support learning of new information. Together, these modules ensure stable knowledge retention and effective task adaptation, even under aggressive quantization. Extensive experiments across multiple benchmarks and bit-width settings demonstrate that AQCL consistently outperforms existing approaches, underscoring its promise for efficient, on-device lifelong learning. Code is available at <https://anonymous.4open.science/r/AQCL-472B/>.

Index Terms—Continual learning, quantization-aware training, adaptive bit allocation, knowledge preserving

I. INTRODUCTION

Quantization has received growing attention as an effective strategy for enabling real-time inference on resource-constrained edge devices [1]–[3]. By converting model parameters into low-bit representations, quantization

significantly reduces memory and computation costs while largely preserving model performance [4]–[7]. While in real-world scenarios, data usually arrives in streams where models must continuously adapt to new tasks without forgetting previous knowledge [8]–[11]. This lifelong adaptation capacity is exactly what continuous learning aims to achieve. However, simply combining quantization and continual learning is far from trivial. Low-bit quantization introduces additional representational distortions that amplify catastrophic forgetting, making it even harder for models to retain past knowledge during sequential learning. These challenges underscore the need for quantization-aware continual learning (QCL) techniques that can preserve prior knowledge, maintain plasticity for new tasks, and remain efficient enough for deployment on edge devices.

Existing low-bit quantization pipelines are primarily designed for static models and rely on quantization-aware training to maintain accuracy [12]–[16]. However, when applied to continual learning, these methods suffer from severe forgetting because *low-bit quantization inherently discards fine-grained information and amplifies feature drift*. Traditional quantization assigns the same limited precision to all layers, regardless of their role in preserving past knowledge. As a result, parameters that are crucial for old tasks are truncated together with less important ones, causing irreversible loss of subtle yet discriminative features. In addition, continual updates introduce feature drift, where representations learned for previous tasks are gradually overwritten by new task parameters. Low-bit quantization further magnifies this effect: the distortion from sequential updates accumulates with the quantization-induced truncation error, pushing representations farther from their original distributions and significantly worsening forgetting.

To address these challenges, we propose Adaptive Quantization for Stable Knowledge Acquisition (AQCL), a unified framework that adaptively allocates layer-wise bit widths to preserve critical knowledge while modulating gradient directions to suppress feature drift. Our Representation-Preserving Quantization (RPQ) module dynamically assigns bit widths according to each layer's importance to previous tasks. We quantify this importance by evaluating the change in the expected loss of past tasks when a standardized perturbation is applied to a given layer. Layers that induce greater loss changes indicating stronger relevance to past knowledge, are allocated higher bit widths to safeguard essential information, while less influential layers remain aggressively compressed. This adaptive strategy preserves previous task representations under low-bit constraints and provides a principled way to

This work was supported in part by the National Key R&D Program of China under Grant No.2023YFA1008600, in part by the National Natural Science Foundation of China under Grant 62576262, Grant U22A2096, and Grant 62376206, in part by the Scientific and Technological Innovation Teams in Shaanxi Province under Grant 2025RS-CXTD-011, in part by the Shaanxi Province Core Technology Research and Development Project under Grant 2024QY2-GJHX-11, in part by the Fundamental Research Funds for the Central Universities under Grant QTZX25083 and Grant QTZX23042, and in part by CCF-Kuaishou Large Model Explorer Fund under Grant CCF-KuaiShou 2025006.

De Cheng, Kun Gu, Lingfeng He and Nannan Wang are with School of Telecommunications Engineering, Xidian University, Xi'an, China (email: dcheng@xidian.edu.cn, kung@stu.xidian.edu.cn, lfhe@stu.xidian.edu.cn, lfhe@stu.xidian.edu.cn, nnwang@xidian.edu.cn). Di Xu is with the Huawei Technologies Co. Ltd (xudi21@huawei.com), Huaijie Wang, Xinbo Gao are with School of Electronic Engineering, Xidian University, Xi'an, China (email: huaijie_wang@stu.xidian.edu.cn, xbgao@mail.xidian.edu.cn).

✉ Corresponding Author: Nannan Wang (nnwang@xidian.edu.cn)

balance accuracy and efficiency.

Furthermore, we introduce a Similarity-Aware Orthogonal Update (SAOU) module that enhances representation stability by selectively modulating gradient directions during new task learning. SAOU decomposes each gradient into two components: one lying within the subspace spanned by previous tasks and the other in its orthogonal complement. Unlike hard null-space projection methods, SAOU does not directly discard the non-null component. Instead, it modulates this component according to its alignment with previous task subspaces, so that useful new task directions can be retained while highly interfering directions are suppressed. The orthogonal component is applied without modification, as it does not interfere with representations learned from earlier tasks. In contrast, the in-subspace component directly perturbs weights critical to previous task features, especially when the gradient direction is highly aligned with the subspace basis. Such perturbations can substantially distort prior representations and lead to pronounced feature drift. To mitigate this, SAOU adaptively scales the projection of the gradient onto each basis vector of the prior subspace, assigning smaller weights to highly aligned directions to prevent overwriting past knowledge, while allowing less-aligned directions to update freely. Through this targeted modulation, SAOU effectively mitigates the drift of old representations while enabling the model to integrate new knowledge in a stable and balanced manner.

Our main contributions are summarized as follows:

- We propose Representation-Preserving Quantization (RPQ), an adaptive quantization module that assigns layer-wise bit widths based on each layer's importance to previous tasks. Importance is measured by how the expected loss of past tasks changes when controlled perturbations are applied to a layer's weights. This enables knowledge-critical layers to retain higher precision, while less influential layers remain aggressively compressed.
- We further introduce Similarity-Aware Orthogonal Update (SAOU), a gradient-modulation strategy that decomposes each update into components lying within prior task subspaces and their orthogonal complement. SAOU preserves the orthogonal component and selectively scales the in-subspace component according to its alignment with the subspace basis, effectively suppressing feature drift while maintaining strong model plasticity.
- We perform comprehensive experiments across diverse bit-width settings and continual learning scenarios. Results demonstrate that our method consistently improves accuracy and substantially reduces forgetting, achieving robust performance across multiple benchmarks.

II. RELATED WORK

Quantization-Aware Training (QAT). QAT embeds quantization criteria into the model training process, where the training loss is computed using the quantized model to guide weight updates [17]–[20]. Existing QAT methods have advanced along several key directions: First, significant efforts focus on designing effective quantizers [12], [21], such as learning optimal clipping thresholds and scaling factors to

preserve critical information even under low-precision constraints. Second, researchers have developed specialized training strategies to stabilize optimization and mitigate accuracy degradation during quantization [22], [23]. Examples include two-stage fine-tuning and knowledge distillation from full-precision teacher models, which help maintain performance under quantization. Third, dynamic quantization techniques [24]–[26] have been introduced to enhance low-bit performance. These methods adaptively adjust quantization ranges based on input statistics or hardware constraints, enabling flexible trade-offs between accuracy and efficiency. Additionally, to address the non-differentiability of quantization functions which is a core challenge in end-to-end training, approximate gradient solutions have been explored [27]–[29]. These approaches use differentiable surrogate functions, allowing quantized networks to be trained with standard gradient-based methods. Despite these advancements, existing QAT frameworks are trained and evaluated exclusively on static, current datasets, failing to account for shifts in task distribution. This oversight limits their applicability to streaming data scenarios, where data distributions evolve dynamically over time.

Continual Learning (CL). CL aims to enable models to learn sequentially from new tasks while retaining prior knowledge, striking a balance between stability (preserving old knowledge) and plasticity (adapting to new tasks) [30]–[32]. Existing CL methods can be categorized into several key families: 1) Regularization-based approaches constrain parameter updates to protect previously learned knowledge [33]–[36]. These methods often estimate parameter importance using metrics like Fisher information or gradient statistics, then penalize changes along critical directions to approximately retain old solutions. 2) Replay-based approaches store or regenerate samples from past tasks and interleave them with current data during training [37]–[45]. This effectively approximates offline joint training, helping reconstruct decision boundaries of previous tasks. 3) Structure-based approaches reduce task interference by adapting or expanding network architectures [46]–[52]. For example, they allocate task-specific modules or dynamically grow model capacity, ensuring different tasks occupy partially isolated parameter subsets. 4) Orthogonal projection techniques guide gradient updates into subspaces orthogonal to representations of prior tasks [53]–[58]. By estimating task-dependent feature or gradient subspaces, these methods prevent future updates from overwriting critical past knowledge, reducing forgetting while maintaining adaptability. Despite these advancements, applying CL directly to quantized models introduces unique challenges. Most existing CL methods are designed for full-precision models and may not perform reliably under the low-bit constraints common in edge deployments.

Quantization-Aware Continual Learning (QCL). To address this limitation, LifeQuant [59] represents an early effort to integrate quantization into continual learning. It leverages exemplar replay and weight regularization to enhance robustness under low-bit deployments, primarily focusing on constraining weight updates and rehearsing a memory buffer while using a largely fixed quantization configuration during training. Although it mitigates forgetting to some degree, it

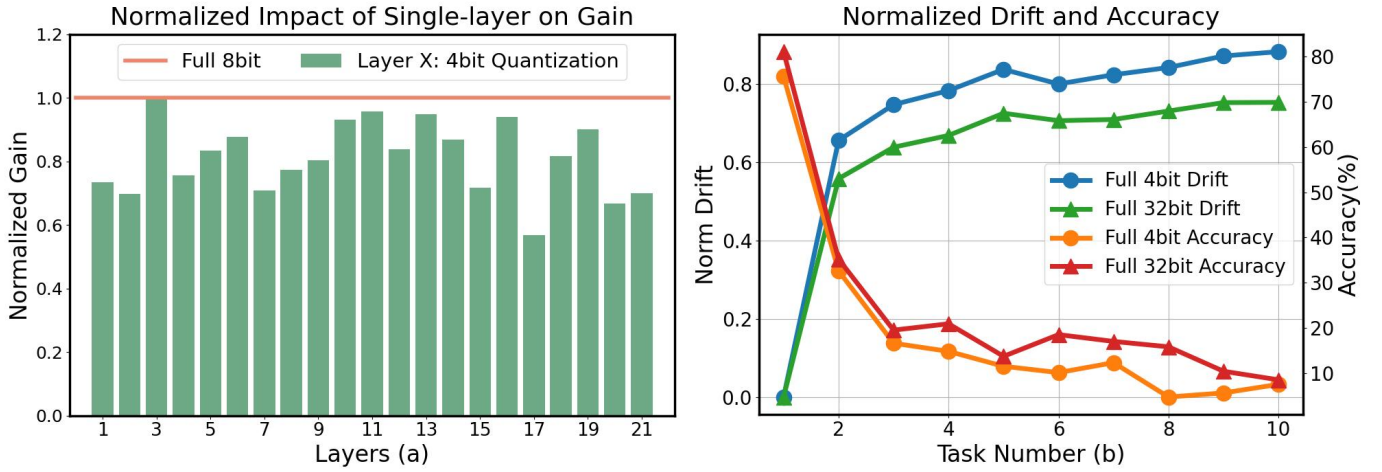


Fig. 1. (a) Normalized gain of applying 4-bit quantization to a single layer of ResNet-20, with the remaining layers quantized to 8-bit. (b) Direct application of 32-bit and 4-bit quantization to continual learning, the drift and the results from the first task to subsequent tasks. All experiments were conducted using ResNet-20 on the CIFAR-100 dataset with 10 tasks.

has key shortcomings: it does not explicitly account for layer-specific impacts of quantization, overlooks representational degradation caused by feature drift, and relies on stored exemplars. This approach may be impractical in strictly memory-constrained scenarios. Beyond LifeQuant, the intersection of quantization and continual learning (QCL) remains relatively underexplored, as most CL methods are still designed for full-precision models. In contrast, our work targets these gaps by introducing two key mechanisms: an adaptive bit-width strategy to preserve parameters in critical layers, and a gradient modulation technique to suppress feature drift. Together, these enable joint optimization of quantization and parameter updates, better balancing stability and plasticity under low-precision constraints.

III. EMPIRICAL ANALYSIS

To evaluate the forgetting issue caused by quantization in continual learning, we explored (1) *the normalized impact of different network layers on model performance under quantization*, and (2) *the impact of quantization on inducing feature drift and how the drift affects performance on previous tasks*. Specifically, we use ResNet-20 [60] model to update the network weights during training in 10-tasks CIFAR-100 [61]. At the same time, given a floating point number x with l and u as its lower and upper bounds respectively, we apply asymmetric uniform quantization [28] $Q(\cdot)$ as follows, where $\text{round}(\cdot)$ denotes the rounding operation and b_l is the quantization bit-width:

$$x^q = Q(x) = \text{round}\left(\frac{x-l}{n}\right) \cdot n + l, \quad n = \frac{u-l}{2^{b_l}-1}. \quad (1)$$

Figure 1 (a) shows the normalized impact of each layer on performance when only a single layer is quantized to 4-bit while the remaining layers are kept at 8-bit. Specifically, the larger the gap between the green bar (Layer X: 4-bit) and the orange reference line (Full 8-bit), the more sensitive that layer is to low-bit quantization, indicating a larger performance

impact. This suggests that the model's performance is more sensitive to quantization at some layers, while at others it shows greater tolerance to low-bit precision. Motivated by this finding, we aim to find a method to quantify the sensitivity of each layer, enabling layer-adaptive bit-width allocation.

Here, the layer indices are assigned following the forward order of the convolutional operators in ResNet-20. They start from the initial convolution and then proceed through the three residual stages, with the projection shortcut convolutions used for downsampling also counted.

Beyond layer-wise quantization effects on performance, we further study the influence of quantization for continual learning on the feature space. Figure 1 (b) presents the feature drift and performance of the initial task across subsequent tasks under both full-precision (32-bit) and 4-bit quantization. Specifically, feature drift is calculated as the L2-norm distance between the features of first task images extracted from models trained after different tasks. We observe that lower accuracy correlates with larger drift, indicating that feature drift is a key factor contributing to forgetting. Compared to the full-precision model, the 4-bit quantized model exhibits consistently higher drift, leading to a more severe performance drop. Inspired by this observation, we aim to reduce quantization-induced drift to alleviate forgetting.

IV. METHOD

A. Problem Definition

For continual learning, let $\{\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_N\}$ define a sequence of N tasks, each associated with a subset dataset $\mathcal{D}_t = \{\mathcal{X}_t, \mathcal{Y}_t\}$, where $\mathcal{X}_t$ and $\mathcal{Y}_t$ denote the input and its corresponding label sets [62], respectively. The training model $\mathbf{f}_{\mathbf{W}}(\cdot)$ is composed of an L -layer neural network, with corresponding model parameters represented as $\mathbf{W} = \{\mathbf{w}_1, \dots, \mathbf{w}_L\}$. During the t -th model training stage, only the current dataset $\mathcal{D}_t$ is accessible. On top of the traditional continual learning, we focus on the quantization-aware continual learning (QCL) [59], with model weights and activations being quantized in limited bit-width (e.g., 2-8 bits) for efficient deployment.

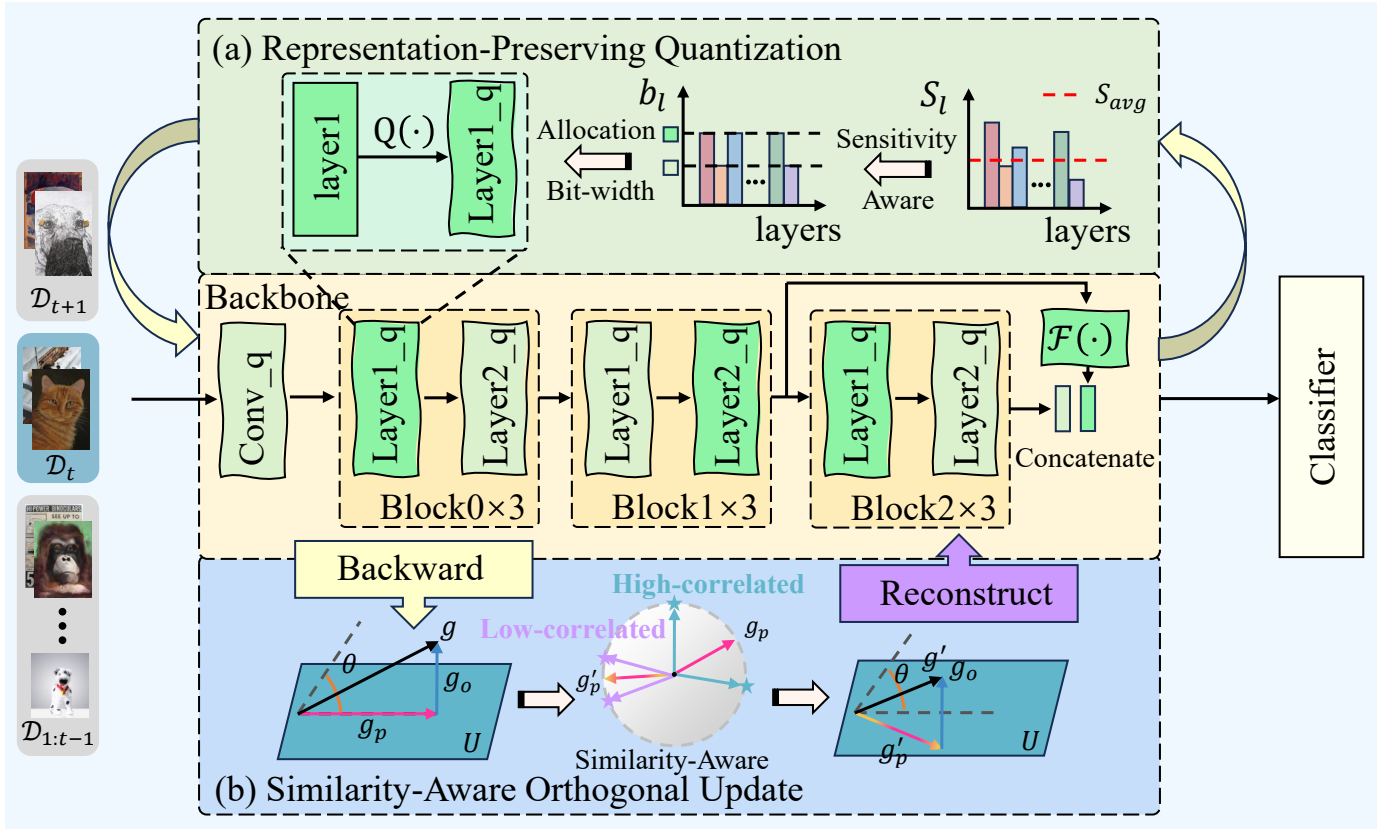


Fig. 2. Overview of our proposed framework. The forward is based on a ResNet-20 backbone. (a) Representation-Preserving Quantization (RPQ) allocates bit-widths for each layer according to its sensitivity. (b) Similarity-Aware Orthogonal Update (SAOU) modulates gradient directions to preserve earlier representations while learning new tasks.

Specifically, the QCL faces two main challenges: 1) How to overcome the performance degradation caused by model quantization, ensuring that the quantized model $\mathbf{f}_{\mathbf{W}^q}(\cdot)$, where $\mathbf{W}^q = Q(\mathbf{W})$, retains the effectiveness of the original model $\mathbf{f}_{\mathbf{W}}(\cdot)$; and 2) how to overcome the severe *catastrophic forgetting* problem of the quantized model on old task data $\mathcal{D}_{t-1}$ after new task learning on $\mathcal{D}_t$. Therefore, the QCL aims to address both of the two above-mentioned issues simultaneously.

B. Method Overview

In this section, we introduce Adaptive Quantization for Stable Knowledge Acquisition (AQCL), a method for QCL that combines two key components: Representation-Preserving Quantization (RPQ) and Similarity-Aware Orthogonal Update (SAOU). Unlike traditional methods that rely on full-precision models, AQCL operates entirely under low-bit constraints. Specifically, RPQ allocates higher bit-widths to layers that are more sensitive to changes in performance on previous tasks, preserving critical layers and compressing less important ones. SAOU mitigates feature drift by keeping the null-space component of the gradient unchanged and modulating the in-subspace component according to its alignment with the prior task subspace, thereby reducing interference on previously learned representations.

Together, RPQ and SAOU allow AQCL to efficiently learn new tasks while maintaining stable performance in low-precision environments.

C. Representation-Preserving Quantization (RPQ)

To address the forgetting from uniform-bit quantization, our RPQ adaptively allocates bit-widths based on layer-wise importance to previous knowledge. It is achieved by a sensitivity estimation algorithm that quantifies the importance, followed by a bit-width allocation rule that assigns appropriate precision accordingly.

Sensitivity estimation. We quantify each layer's sensitivity by measuring the expected increase in the previous tasks' loss when a fixed perturbation is injected into that layer's weights or activations. This provides a direct estimate of how strongly quantization noise in that layer would affect performance. The same derivation applies to activations, by substituting weights with activations and perturbing them accordingly, we obtain an analogous activation-level sensitivity measure. For clarity of exposition, we focus on the weight-based formulation below.

Definition 1: Given a perturbation $\|\Delta \mathbf{w}_l^q\|_2^2$ for layer L_l , the sensitivity of a layer is defined as $S_l = \mathbb{E}[\Delta \mathcal{L}_l] / (\|\Delta \mathbf{w}_l^q\|_2^2)$, where $\mathbb{E}[\cdot]$ denotes the expectation operator. Consider two layers L_1 and L_2 , we define the perturbation as independently sampled from $\mathcal{N}(0, \Sigma)$:

$$\|\Delta \mathbf{w}_1^q\|_2^2 = \|\Delta \mathbf{w}_2^q\|_2^2, \quad \Delta \mathbf{w}_1^q \in \mathbb{R}^{n_1 \times d_1}, \quad (2)$$

where $\Delta \mathbf{w}_1^q$ and $\Delta \mathbf{w}_2^q$ denote perturbations to the weight matrices of layers L_1 and L_2 , respectively. In particular, $\Delta \mathbf{w}_1^q \in \mathbb{R}^{n_1 \times d_1}$ is an isotropic random perturbation, where n_1 and d_1 are the numbers of output and input channels of layer L_1 .

We implement the sensitivity estimation after training each task. For the L_1 layer, we first flatten the quantized weight and its perturbation as $\hat{\mathbf{w}}_1^q = \text{vec}(\mathbf{w}_1^q)$ and $\Delta \hat{\mathbf{w}}_1^q = \text{vec}(\Delta \mathbf{w}_1^q)$. After training task t , we perform the Taylor expansion around the quantized weights and evaluate the local loss change on the current task data:

$$\begin{aligned} \mathcal{L}_1(\hat{\mathbf{w}}_1^q + \Delta \hat{\mathbf{w}}_1^q) &= \mathcal{L}_1(\hat{\mathbf{w}}_1^q) + \nabla \mathcal{L}_1(\hat{\mathbf{w}}_1^q)^\top \Delta \hat{\mathbf{w}}_1^q \\ &\quad + \frac{1}{2} \Delta \hat{\mathbf{w}}_1^{q\top} \mathbf{H}_1 \Delta \hat{\mathbf{w}}_1^q + o(\|\Delta \hat{\mathbf{w}}_1^q\|^2). \end{aligned} \quad (3)$$

Since the sensitivity is computed after training the current task, the Taylor expansion is performed around the current task solution. Thus, the gradient term is locally small on the current task data [63], [64]. In addition, because the perturbation is zero-mean, the expectation of the first-order term $\nabla \mathcal{L}_1(\hat{\mathbf{w}}_1^q)^\top \Delta \hat{\mathbf{w}}_1^q$ is zero:

$$\mathbb{E}[\nabla \mathcal{L}_1(\hat{\mathbf{w}}_1^q)^\top \Delta \hat{\mathbf{w}}_1^q] = \nabla \mathcal{L}_1(\hat{\mathbf{w}}_1^q)^\top \mathbb{E}[\Delta \hat{\mathbf{w}}_1^q] = 0. \quad (4)$$

After omitting the higher-order term $o(\|\Delta \hat{\mathbf{w}}_1^q\|^2)$, the expected loss increase is approximated as:

$$\mathbb{E}[\Delta \mathcal{L}_1] \approx \mathbb{E}\left[\frac{1}{2} \Delta \hat{\mathbf{w}}_1^{q\top} \mathbf{H}_1 \Delta \hat{\mathbf{w}}_1^q\right]. \quad (5)$$

Since the Hessian matrix $\mathbf{H}_1$ is a square matrix, we can transform $\Delta \mathcal{L}_1$ into the following based on the trace identity:

$$\begin{aligned} \mathbb{E}[\Delta \mathcal{L}_1] &= \frac{1}{2} \text{Tr}(\mathbb{E}[\Delta \hat{\mathbf{w}}_1^q \Delta \hat{\mathbf{w}}_1^{q\top}] \mathbf{H}_1) \\ &= \frac{1}{2} \text{Tr}(\mathbf{\Sigma}_1 \mathbf{H}_1), \end{aligned} \quad (6)$$

where $\text{Tr}(\cdot)$ denotes the trace operator. Since $\Delta \mathbf{w}_1^q$ is a random perturbation in all directions and is sampled from $\mathcal{N}(0, \mathbf{\Sigma}_1)$, its mean is $\mathbb{E}[\Delta \hat{\mathbf{w}}_1^q] = 0$, and we denote the covariance as $\mathbf{\Sigma}_1$.

For independent, identically distributed perturbations with variance λ_1 , we simplify Eq. (6) as follows:

$$\mathbf{\Sigma}_1 = \lambda_1 \mathbf{I}_{n_1 d_1 \times n_1 d_1} \Rightarrow \mathbb{E}[\Delta \mathcal{L}_1] = \frac{1}{2} \lambda_1 \text{Tr}(\mathbf{H}_1). \quad (7)$$

Here, $\mathbf{I}_{n_1 d_1 \times n_1 d_1}$ denotes the identity matrix of size $n_1 d_1 \times n_1 d_1$. Similarly, we transform the perturbation magnitude $\|\Delta \mathbf{w}_1^q\|_2^2$ based on the trace identity as follows:

$$\|\Delta \mathbf{w}_1^q\|_2^2 = \|\Delta \hat{\mathbf{w}}_1^q\|_2^2 = \text{Tr}(\mathbf{\Sigma}_1) = n_1 d_1 \lambda_1. \quad (8)$$

By substituting Eq. (8) into Eq. (7), we obtain the sensitivity of a layer as follows:

$$\frac{\mathbb{E}[\Delta \mathcal{L}_1]}{\|\Delta \mathbf{w}_1^q\|_2^2} = \frac{1}{2} \cdot \frac{\text{Tr}(\mathbf{H}_1)}{n_1 d_1}. \quad (9)$$

The Gaussian distribution is not a necessary assumption in the above derivation. The expected loss increase depends on the perturbation through its covariance. For the uniform quantizer in Eq. (1), the round-off error can also be modeled as zero-mean uniform noise. Since the quantization step

size is defined as n in Eq. (1), each perturbation element has variance $n^2/12$. Thus, under the isotropic setting, the covariance becomes $\mathbf{\Sigma}_1 = \frac{n^2}{12} \mathbf{I}_{n_1 d_1 \times n_1 d_1}$. After normalization by the perturbation magnitude in Eq. (8), it yields the same sensitivity form as Eq. (9). Therefore, the layer sensitivity is still governed by the normalized Hessian trace under the standard uniform round-off noise model.

The above formula demonstrates that, under a fixed perturbation, the loss variation of a layer is governed by its normalized Hessian trace:

$$\mathbb{E}[\Delta \mathcal{L}_1] \leq \mathbb{E}[\Delta \mathcal{L}_2] \Leftrightarrow \frac{1}{2n_1 d_1} \text{Tr}(\mathbf{H}_1) \leq \frac{1}{2n_2 d_2} \text{Tr}(\mathbf{H}_2). \quad (10)$$

Therefore, layers with a larger normalized Hessian trace are more sensitive to the same perturbation, and we define the sensitivity as:

$$S_l = \frac{1}{2n_l d_l} \text{Tr}(\mathbf{H}_l). \quad (11)$$

In practice, directly obtaining the exact Hessian trace requires additional second-order computation. Computing and storing the full Hessian matrix is also prohibitive. For the l -th layer with $n_l d_l$ parameters, the full Hessian requires $O((n_l d_l)^2)$ storage. In RPQ, we only need the Hessian trace in Eq. (11), where $\text{Tr}(\mathbf{H}_l) = \sum_i H_{l,ii}$. Therefore, the off-diagonal entries, although they describe cross-parameter correlations, do not directly enter the sensitivity score S_l . We approximate this trace using the diagonal empirical Fisher:

$$\text{Tr}(\mathbf{H}_l) \approx \text{Tr}(\mathbf{F}_l) = \sum_{i,j} \mathbb{E} \left[\left(\frac{\partial \mathcal{L}_l}{\partial w_{l,ij}^q} \right)^2 \right]. \quad (12)$$

Thus, the diagonal empirical Fisher is used to estimate the trace required by RPQ, rather than to recover the full Hessian matrix. Since the diagonal empirical Fisher can be obtained from first-order gradients, it avoids additional second-order computation and full-Hessian storage while keeping the sensitivity estimation efficient. Fisher-based estimates are also commonly used to measure parameter importance and approximate local curvature in continual learning [63], [64]. The estimated layer-wise sensitivities are then utilized to guide the adaptive bit-width allocation.

Bit-width allocation rule. We design the following rule to adaptively allocate the quantization bit-width for each layer: layers whose sensitivity to loss variations is greater than or equal to the average sensitivity S_{avg} are assigned high bit-widths b^H , and the remaining layers are assigned low bit-widths b^L . This can be expressed as:

$$b_l = \begin{cases} b^H, & \text{if } S_l \geq S_{\text{avg}}, \\ b^L, & \text{if } S_l < S_{\text{avg}}, \end{cases} \quad (13)$$

where S_l is the sensitivity of layer l , b_l denotes the bit-width of layer l , and b^H and b^L represent the high and low quantization bit-widths. The rule in Eq. (13) is applied independently to weights and activations. For the weights of layer l , we perturb

Algorithm 1 RPQ: Representation-Preserving Quantization

Inputs: Dataset $\mathcal{D}_t = \{\mathcal{X}_t, \mathcal{Y}_t\}$ for $\mathcal{T}_t \in \{\mathcal{T}_1, \dots, \mathcal{T}_N\}$; network $\mathbf{f}_W$ with L layers.

Outputs: Updated quantized weights $\mathbf{w}_l^q$ and subspaces $\mathbf{U}_o, \mathbf{U}_p$.

- 1: **Initialization:** Initialize $\mathbf{W}$ and learning rate η ; Insert $\mathcal{F}(\cdot)$ at an intermediate layer and concatenate with the final representation.
- 2: **for** $\mathcal{T}_t$ **do**
- 3: **for** $l = 1, \dots, L$ **do**
- 4: **if** $t = 1$ **then**
- 5: Sample $(\mathbf{x}, \mathbf{y}) \sim (\mathcal{X}_1, \mathcal{Y}_1)$;
- 6: Train with $\mathbf{x}^q = Q(\mathbf{x})$, $w_l^q = Q(w_l)$ to convergence;
- 7: // Sensitivity estimation
- 8: Compute $S_l = \frac{1}{2n_l d_l} \text{Tr}(\mathbf{H}_l)$ (Eq. (11));
- 9: Compute $\mathbf{U}_o$ and $\mathbf{U}_p$ by SVD (Eq. (20));
- 10: **else**
- 11: Initialize the quantized weight of layer l using the previous task's quantized weight and $s = 0$;
- 12: **while** not converged **do**
- 13: Sample $(\mathbf{x}, \mathbf{y}) \sim (\mathcal{X}_t, \mathcal{Y}_t)$;
- 14: // Adaptive quantization by S_l
- 15: $\mathbf{x}^q = Q(\mathbf{x})$, $w_l^q = Q(w_l)$ (Eq. (1));
- 16: Train on $\mathcal{T}_t$ to obtain $\mathbf{g}$;
- 17: // Similarity-aware orthogonal update
- 18: $\Delta w_l^q \leftarrow \text{SAOU}(\mathbf{g}, \mathbf{U}_o, \mathbf{U}_p, \theta, \alpha)$;
- 19: $\mathbf{w}_{l,s+1}^q \leftarrow \mathbf{w}_{l,s}^q - \eta \Delta w_{l,s}^q$;
- 20: $s \leftarrow s + 1$;
- 21: **end while**
- 22: Compute the layer sensitivity S_l by Eq. (11);
- 23: Collect $\mathbf{Z}_t^l$ and update the covariance $\mathbf{X}_t^l$;
- 24: Perform SVD on $\mathbf{X}_t^l$;
- 25: Update $\mathbf{U}_o^l$ and $\mathbf{U}_p^l$ by the SAOU;
- 26: **end if**
- 27: **end for**
- 28: **end for**

the quantized weight $\mathbf{w}_l^q$ and compute the weight sensitivity as:

$$S_l^w = \frac{1}{2n_l^w d_l^w} \text{Tr}(\mathbf{H}_l^w), \quad (14)$$

where $n_l^w d_l^w$ is the number of weight parameters. The Hessian trace is approximated by the empirical Fisher using the gradients with respect to $\mathbf{w}_l^q$. For activations, we use the same formulation by perturbing the quantized activation $\mathbf{a}_l^q$:

$$S_l^a = \frac{1}{2n_l^a d_l^a} \text{Tr}(\mathbf{H}_l^a), \quad (15)$$

where $n_l^a d_l^a$ is the number of activation elements. The trace of $\mathbf{H}_l^a$ is estimated using the gradients with respect to $\mathbf{a}_l^q$. After obtaining S_l^w and S_l^a , we apply Eq. (13) to them separately. This gives the weight bit-width b_l^w and the activation bit-width b_l^a for each layer.

However, the empirical Fisher is still an approximation to the Hessian diagonal. As a result, the estimated trace may differ from the exact Hessian trace. Because RPQ defines layer

sensitivity using the normalized Hessian trace and compares each layer sensitivity with the average sensitivity for bit-width allocation, the potential effect of this approximation is mainly on layers close to the allocation threshold, while layers farther from the threshold are less likely to change their assigned bit-width. This reflects the trade-off between Hessian-trace estimation accuracy and computational efficiency in RPQ.

Although adaptive bit-width allocation better preserves the knowledge in sensitive layers, quantization-induced truncation can still degrade deep feature representations. We address this by inserting a convolution layer $\mathcal{F}(\cdot)$ into an intermediate layer and concatenating its output with the final representation, enabling mid-level semantics to reinforce deep representations at lower overhead.

Such a sensitivity-aware bit-width allocation paradigm effectively mitigates the forgetting from quantization in continual learning. By assigning higher bit-widths to sensitive layers, we protect important historical knowledge during quantization, while simultaneously compressing redundant parameters for efficient deployment. This mechanism enables anti-forgetting, low-precision continual learning on resource-constrained devices. Implementation is summarized in Alg. 1.

D. Similarity-Aware Orthogonal Update (SAOU)

To mitigate feature drift amplified by low-bit quantization, we propose the **Similarity-Aware Orthogonal Update (SAOU)** module, which follows idea of projection-based continual learning. However, low-bit quantization already limits the representation capacity of the model and its flexibility in learning new tasks. Under this additional limitation, directly constraining the update to the approximate null space may be overly conservative, since the discarded non-null component may still contain useful gradient information for current task learning. Therefore, SAOU does not directly discard the non-null component. Instead, it selectively modulates this component according to its alignment with previous task subspaces, suppressing highly aligned directions while retaining less interfering directions for current task learning.

Specifically, we decompose the update gradient $\mathbf{g}$ into two components: one that falls in its null space $\mathbf{g}_o$, preserves previously learned knowledge and does not cause significant drift; and the other that lies in the prior tasks' input subspace $\mathbf{g}_p$, allows directions with lower alignment to freely update without significantly interfering with prior representations:

$$\mathbf{g} = \mathbf{g}_o + \mathbf{g}_p. \quad (16)$$

The orthogonal update component $\mathbf{g}_o$, which corresponds to the part of the gradient that does not affect prior representations, can be derived from the following lemma [53]:

Lemma 1: Given the null space of the uncentered feature covariance matrix $\mathbf{X}_{t-1}$ and parameters $\mathbf{w}_{t-1}$, both constructed from all tasks prior to task t , the update $\mathbf{g}_t$ for the new task t will not interfere with the representations learned for previous tasks if:

$$\mathbf{X}_{t-1}^\top \cdot \mathbf{g}_t = 0. \quad (17)$$

Then, for any previous task s ($s < t$), the feature output remains unchanged:

$$\mathcal{F}(\mathbf{X}_s, \mathbf{w}_s) = \mathcal{F}(\mathbf{X}_s, \mathbf{w}_t), \quad (18)$$

where $\mathbf{w}_t = \mathbf{w}_{t-1} + \mathbf{g}_t$ denotes the updated parameters for task t .

The orthogonal update component $\mathbf{g}_o$ can be written as follows:

$$\mathbf{g}_o = \mathbf{U}_o \mathbf{U}_o^\top \mathbf{g}, \quad (19)$$

where $\mathbf{U}_o$ denotes the set of singular vectors associated with the smallest singular values of $\mathbf{X}_{t-1}$. These vectors are selected using a threshold λ : specifically, we retain those whose singular values are less than or equal to λ times the smallest singular value of $\mathbf{X}_{t-1}$. The singular value matrix Σ and the corresponding singular vectors $\mathbf{U}$ can be obtained through Singular Value Decomposition (SVD), expressed as:

$$\mathbf{X}_{t-1} = \mathbf{U} \Sigma \mathbf{U}^\top. \quad (20)$$

In practice, the subspaces $\mathbf{U}_o^l$ and $\mathbf{U}_p^l$ are maintained in a task-wise manner. Before learning task t , they are obtained from the accumulated covariance $\mathbf{X}_{t-1}^l$ of previous tasks. After finishing task t , we forward the current task data through the trained model and collect the input features of the l -th layer, denoted as $\mathbf{Z}_t^l$. The current task covariance is computed as:

$$\mathbf{X}_{t,\text{cur}}^l = \frac{1}{n_t} (\mathbf{Z}_t^l)^\top \mathbf{Z}_t^l, \quad (21)$$

where n_t is the number of samples in task t . To avoid storing previous task samples or historical features, we update the accumulated covariance by:

$$\mathbf{X}_t^l = \frac{\bar{n}_{t-1}}{\bar{n}_t} \mathbf{X}_{t-1}^l + \frac{n_t}{\bar{n}_t} \mathbf{X}_{t,\text{cur}}^l, \quad \bar{n}_t = \bar{n}_{t-1} + n_t. \quad (22)$$

Then, SVD is performed on $\mathbf{X}_t^l$. Following the subspace selection rule in SAOU, $\mathbf{U}_o^l$ is selected from the singular vectors with small singular values, and $\mathbf{U}_p^l$ is selected from the transition singular-value range. The obtained subspaces are fixed during the learning of task $t+1$ and updated again after that task is completed.

The switching rule determines when the SAOU should be used. We use the cosine similarity between the candidate gradient $\mathbf{g}$ and the complementary subspace $\bar{\mathbf{U}}_o$ as the criterion. A larger similarity means that $\mathbf{g}$ contains a clear component outside the approximate null space, which can provide useful directions for learning the current task but may also disturb old task representations. When the similarity is larger than θ , SAOU decomposes $\mathbf{g}$ into $\mathbf{g}_o$ and $\mathbf{g}_p$, and applies the similarity-aware mask to $\mathbf{g}_p$ before reconstructing the final update. This allows the model to retain useful new task information while suppressing directions highly aligned with old task subspaces. When the similarity is not larger than θ , the non-null-space component is weak, and SAOU adopts the more conservative orthogonal update by only keeping $\mathbf{g}_o$.

The similarity-aware component $\mathbf{g}_p$ lies within the subspace that primarily drives the learning of new tasks. To prevent excessive updates in dominant directions, we rescale $\mathbf{g}_p$ within a transition subspace $\mathbf{U}_p = [\mathbf{u}_{p1}, \mathbf{u}_{p2}, \dots, \mathbf{u}_{pk}]$, which lies

Algorithm 2 SAOU: Similarity-Aware Orthogonal Update

Inputs: Candidate gradient $\mathbf{g}$; null space $\mathbf{U}_o$; complementary subspace $\bar{\mathbf{U}}_o$; transition subspace $\mathbf{U}_p$; thresholds θ, α .

Outputs: Updated gradient Δw_l^q .

- 1: **Initialization (for $t=1$):** Compute the null space $\mathbf{U}_o$ and the transition subspace $\mathbf{U}_p$ by SVD;
- 2: **if** $|\cos(\mathbf{g}, \bar{\mathbf{U}}_o)| > \theta$ **then**
- 3: Decompose gradient:
 $\mathbf{g}_o = \mathbf{U}_o \mathbf{U}_o^\top \mathbf{g}$, $\mathbf{g}_p = \mathbf{g} - \mathbf{g}_o$;
 // Compute similarity mask
- 4: $\mathbf{M}_{ij} = \begin{cases} \exp(-\alpha \cos^2(\mathbf{g}_p, \mathbf{u}_{pi})), & i = j, \\ 0, & i \neq j, \end{cases}$ (Eq. (23));
 // Reconstruct similarity-aware component
- 5: $\mathbf{g}_p' = \mathbf{U}_p \mathbf{M} (\mathbf{U}_p \mathbf{M})^\top \mathbf{g}_p$ (Eq. (24));
- 6: **Return** $\Delta w_l^q = \mathbf{g}_o + \mathbf{g}_p'$;
- 7: **else**
- 8: **Return** $\Delta w_l^q = \mathbf{U}_o \mathbf{U}_o^\top \mathbf{g}$;
- 9: **end if**

between the approximate null space and the principal subspace and is likewise obtained from the SVD of $\mathbf{X}_{t-1}$. This design ensures $\mathbf{g}_p$ is not overly concentrated along the most sensitive directions, but is instead directed through a more moderate region.

Specifically, we compute the similarity between $\mathbf{g}_p$ and each basis direction in $\mathbf{U}_p$ to determine how much each direction should contribute to the update. Based on these similarities, SAOU reconstructs $\mathbf{g}_p$ using a diagonal similarity mask $\mathbf{M}$:

$$\mathbf{M}_{ij} = \begin{cases} \exp(-\alpha \cos^2(\mathbf{g}_p, \mathbf{u}_{pi})), & i = j, \\ 0, & i \neq j, \end{cases} \quad (23)$$

Here, $\mathbf{M}_{ij}$ denotes the i, j -th element of $\mathbf{M}$, $\cos(\mathbf{g}_p, \mathbf{u}_{pi}) = \mathbf{g}_p^\top \mathbf{u}_{pi} / (\|\mathbf{g}_p\| \cdot \|\mathbf{u}_{pi}\|)$ represents the cosine similarity between $\mathbf{g}_p$ and each basis vector $\mathbf{u}_{pi}$, and α is a hyperparameter controlling the strength of suppression. This mask adjusts each component according to its alignment with prior task directions. Directions with higher similarity are suppressed to avoid interference with previous knowledge, while those with lower similarity are promoted to support learning of the new task. Therefore, the final update becomes:

$$\mathbf{g}' = \mathbf{g}_o + \mathbf{U}_p \mathbf{M} (\mathbf{U}_p \mathbf{M})^\top \mathbf{g}_p. \quad (24)$$

The SAOU module decomposes the gradient into orthogonal and similarity-aware components: the orthogonal part resides in an approximate null space, preserving previously learned knowledge and preventing significant drift, while the similarity-aware part lies in the input subspace of prior tasks, allowing updates in directions with lower alignment to update freely without significantly interfering with prior representations. This ensures that directions with higher alignment are suppressed, mitigating feature drift amplified by quantization while retaining adaptability for new tasks. Implementation is summarized in Alg. 2.

TABLE I
COMPARISON OF ACC (%) AND BWT (%) UNDER 8-BIT, 4-BIT, AND 2-BIT QUANTIZATION ON CIFAR100 AND TINYIMAGENET WITH VARYING NUMBER OF TASKS.

Bit-width	Method	CIFAR100				TinyImageNet	
		$T = 10$		$T = 20$		$T = 25$	
		ACC (%)	BWT (%)	ACC (%)	BWT (%)	ACC (%)	BWT (%)
32-bit	Ours	64.12	-5.71	63.75	-8.38	44.31	-11.08
8-bit	Vanilla-QCL	23.74	-61.72	23.41	-67.71	15.71	-53.63
	EWC [63]	56.44	-2.17	57.89	-5.55	36.78	-5.85
	GPM [65]	44.76	-8.42	50.91	-10.68	—	—
	TRGP [66]	45.76	-3.14	51.63	-15.03	—	—
	Adam-NSCL [53]	59.80	-4.32	59.38	-6.22	41.70	-6.96
	Adam-NSCL w/ $\mathcal{F}(\cdot)$ [53]	60.75	-5.40	62.01	-6.63	42.65	-7.30
	AdNS [67]	59.45	-6.59	62.02	-9.88	42.10	-5.06
	AdNS w/ $\mathcal{F}(\cdot)$ [67]	60.83	-7.67	62.66	-10.12	43.27	-6.93
	Ours	63.96	-5.39	63.56	-8.60	44.22	-11.98
4-bit	Vanilla-QCL	21.78	-69.27	22.56	-67.45	14.56	-59.67
	EWC [63]	55.51	-2.96	57.44	-4.52	35.03	-3.93
	GPM [65]	43.36	-2.94	41.06	-6.23	—	—
	TRGP [66]	44.06	-3.82	41.09	-8.26	—	—
	Adam-NSCL [53]	56.32	-2.79	58.88	-8.87	36.78	-5.36
	Adam-NSCL w/ $\mathcal{F}(\cdot)$ [53]	57.19	-3.19	60.27	-9.74	37.11	-6.14
	AdNS [67]	57.06	-7.44	61.78	-8.89	37.28	-4.86
	AdNS w/ $\mathcal{F}(\cdot)$ [67]	58.18	-7.78	62.16	-9.71	38.11	-5.35
	Ours	63.84	-5.26	63.38	-10.88	43.96	-9.85
2-bit	Vanilla-QCL	20.74	-55.12	21.40	-56.71	13.27	-52.14
	EWC [63]	44.18	-8.63	48.55	-14.42	27.08	-2.24
	GPM [65]	36.16	-2.47	40.88	-3.51	—	—
	TRGP [66]	35.94	-1.76	40.58	-3.80	—	—
	Adam-NSCL [53]	45.40	-1.32	51.33	-3.89	31.89	-8.69
	Adam-NSCL w/ $\mathcal{F}(\cdot)$ [53]	47.24	-2.21	53.85	-4.44	33.23	-9.17
	AdNS [67]	45.45	-4.15	54.78	-5.96	32.92	-3.60
	AdNS w/ $\mathcal{F}(\cdot)$ [67]	47.49	-5.12	55.39	-6.53	34.22	-8.41
	Ours	51.19	-5.88	58.04	-6.68	35.41	-11.81

V. EXPERIMENTS

A. Experimental Details

Datasets. We evaluate our method on two benchmark datasets under the quantization-aware continual learning setting: CIFAR-100 [61] and TinyImageNet [68]. CIFAR-100 consists of 60,000 images across 100 classes. TinyImageNet contains 100,000 images from 200 classes. Following prior works [53] and ensuring a fair comparison, we adopt the following task configurations: CIFAR-100 is split into 10 tasks with 10 classes each and 20 tasks with 5 classes each; TinyImageNet is divided into 25 tasks with 8 classes each.

Evaluation Metrics. Following prior works [53] in continual learning, we adopt two widely used metrics to evaluate performance in quantization-aware continual learning: 1). Average Accuracy (ACC), which measures the mean test accuracy across all tasks after the final task has been learned. 2). Backward Transfer (BWT), which assesses the impact of learning new tasks on the performance of previously learned tasks. The metrics are formally defined as $ACC = \frac{1}{N} \sum_{t=1}^N A_{N,t}$, $BWT = \frac{1}{N-1} \sum_{t=1}^{N-1} (A_{N,t} - A_{t,t})$, where $A_{N,t}$ is the test accuracy on the t -th task after training sequentially on all N tasks. Both ACC and BWT are the larger the better. A high ACC is more important than a high BWT under the same conditions, because ACC represents the overall

performance of the network stability and plasticity, while BWT only reflects the stability.

Implementation Details. All experiments are implemented in PyTorch [69] and conducted on a single NVIDIA RTX 3090 GPU. We adopt ResNet-20 as the backbone network, with quantization applied to all trainable layers, and the high bit-width b^H is set to $2b^L$ (e.g., when $b^L=8$ -bit, $b^H=16$ -bit). During training, all models are optimized using the Adam optimizer with a batch size of 32. The training schedule spans 80 epochs, with learning rate decay at the 30th, 60th, and 80th epochs. Following the protocol of [53], the learning rates for different components are set as follows: $1e-4$ for the model parameters, $1e-3$ for the classification head, and $5e-4$ for batch normalization layers. Weight decay for model parameters is set to $5e-5$. After each task, we compute the SVD of the covariance matrix and define the approximate null space by retaining singular vectors whose singular values are below the threshold $\lambda = 10$, consistent with [53]. The SAOU transition subspace U_p is formed by singular vectors with singular values in the range $[0.5\lambda, 2\lambda]$, and its use is controlled by a threshold θ .

B. Experimental Results

Comparison with Continual Learning Methods under Quantization. The proposed method consistently outperforms

TABLE II
ABLATION STUDY OF RPQ AND SAOU MODULES UNDER 4-BIT AND 2-BIT QUANTIZATION.

Idx	RPQ	SAOU	Bit-width	CIFAR100				TinyImageNet	
				$T = 10$		$T = 20$		$T = 25$	
				ACC (%)	BWT (%)	ACC (%)	BWT (%)	ACC (%)	BWT (%)
1	–	–	4-bit	21.78	-69.27	22.56	-67.45	14.56	-59.67
2	✓	–		29.53	-58.22	28.14	-62.00	15.12	-58.32
3	–	✓		57.39	-6.11	59.25	-9.94	37.06	-9.24
4	✓	✓		63.84	-5.26	63.38	-10.88	43.96	-9.85
1	–	–	2-bit	20.74	-55.12	21.40	-56.71	13.27	-52.14
2	✓	–		23.87	-53.35	27.57	-54.67	14.77	-51.26
3	–	✓		45.94	-4.46	52.83	-6.04	32.32	-9.10
4	✓	✓		51.19	-5.88	58.04	-6.68	35.41	-11.81

state-of-the-art (SOTA) methods across various benchmarks, as shown in Table I. We evaluate our method against other continual learning methods [53], [63], [65]–[67] under quantization. Vanilla-QCL denotes the quantization-aware continual learning baseline without RPQ or SAOU. For fair comparisons, all methods are evaluated under the same quantization settings and use ResNet-20 as the base network. Table I presents experimental results on CIFAR-100 and TinyImageNet datasets with varying task sequence lengths ($T = 10, 20, 25$). Notably, under the same tuning conditions, our method consistently outperforms continual learning approaches in the quantization setting, highlighting its effectiveness in mitigating catastrophic forgetting under quantization constraints.

The first four columns of Table I show the quantization results for full precision (32-bit), 8-bit, 4-bit and 2-bit ResNet-20 on CIFAR-100 under two different task configurations. For example, our approach based on 4-bit of 10-tasks CIFAR-100 enhances the accuracy by 6% to 20%. Moreover, its performance is comparable to that of 8-bit and even approaches 32-bit performance, which further demonstrates the effectiveness of our method. These results demonstrate that our proposed method consistently outperforms all baselines, validating the effectiveness of our representation-preserving quantization and similarity-aware update in addressing the challenges of catastrophic forgetting under low-bit constraints.

To make the comparison more controlled, we add the same additional convolutional branch $\mathcal{F}(\cdot)$ to Adam-NSCL and AdNS. The branch is inserted at the same intermediate layer and concatenated with the final representation, while all other experimental settings are kept unchanged.

As shown in Table I, adding $\mathcal{F}(\cdot)$ improves the competitive baselines, confirming that mid-level features are useful under low-bit quantization. However, AQCL still achieves better overall performance, suggesting that the improvement comes not only from $\mathcal{F}(\cdot)$, but also from RPQ and SAOU, which consider quantization sensitivity and representation drift.

We also observe that the BWT of Adam-NSCL and AdNS decreases after adding the same $\mathcal{F}(\cdot)$ branch. This is because $\mathcal{F}(\cdot)$ enhances the representation capacity and thus increases the plasticity for current task learning. However, stronger plasticity may also increase the risk of representation changes during later updates [70]. This observation suggests that $\mathcal{F}(\cdot)$ mainly enhances representation learning, while its contribution is more related to plasticity than to stability.

C. Ablation Study

The effectiveness of RPQ and SAOU. Our approach introduces two key innovations: 1) Representation-Preserving Quantization (RPQ) and 2) Similarity-Aware Orthogonal Update (SAOU). To evaluate the effectiveness of these proposed components, we conduct an ablation study across multiple tasks on CIFAR-100 and TinyImageNet, as shown in Table II.

The Baseline model (Idx1) implements standard quantization in the continual learning setting with the ResNet-20 backbone. In Idx2, the RPQ module is integrated into the baseline. Experimental results show consistent improvements in both ACC and BWT across both datasets, demonstrating the effectiveness of RPQ in preserving critical layer knowledge and mitigating quantization-induced forgetting. In Idx3, the SAOU module is incorporated into the baseline. The results confirm that SAOU improves new task learning and preserves old knowledge by modulating gradient directions and reducing feature drift from previous tasks, without significantly hindering plasticity. Idx4 combines both RPQ and SAOU, and the results demonstrate that their combination achieves the best performance.

Experiment on Precision Allocation. To validate the design choice of sensitivity-based bit-width assignment in the proposed RPQ module, we conduct an experiment in 10-tasks CIFAR-100, where only half of the layers are allowed to use high-precision (8-bit) quantization. As shown in Table III, we compare our sensitivity-based bit-width allocation against a random assignment baseline on old task average accuracy, with full 8-bit and full 4-bit models serving as upper and lower bounds, respectively. In our method, high bits are assigned to layers with above-average sensitivity, estimated via their normalized Hessian trace approximated by Fisher information. The results show that while higher precision allocation can yield some improvement, sensitivity-based assignment achieves better performance by dedicating limited resources to the most sensitive layers. This supports our RPQ design: adaptively allocating bit-widths based on layer-wise sensitivity ensures critical layers are preserved while redundant layers are compressed, particularly in resource-constrained continual learning settings.

Analysis of Diagonal Similarity Mask Strategies. To evaluate the impact of different diagonal similarity mask strategies within SAOU, we analyze the performance of our

TABLE III

AVERAGE ACCURACY ON OLD TASKS IN 10-TASKS CIFAR-100 UNDER 4-BIT QUANTIZATION. OURS IS COMPARED WITH RANDOM, FULL 8-BIT, AND FULL 4-BIT SETTINGS.

Method	Task 2	Task 3	Task 4	Task 5	Task 6	Task 7	Task 8	Task 9	Task 10
Full 4bit	67.10	59.50	59.53	59.93	56.90	57.62	57.91	56.65	58.36
Random	68.50	61.45	60.77	61.58	57.50	59.60	60.21	57.04	61.10
Ours	70.40	62.65	61.80	62.10	57.66	59.92	60.61	58.96	62.44
Full 8bit	73.30	66.20	62.57	63.38	60.76	60.73	61.16	58.95	62.52

TABLE IV

COMPARISON OF ACC (%) AND BWT (%) UNDER 4-BIT QUANTIZATION IN 10-TASKS CIFAR-100 AND 25-TASKS TINYIMAGENET.

Method	CIFAR100		TinyImageNet	
	ACC (%)	BWT (%)	ACC (%)	BWT (%)
Orthogonal-Only	62.65	-1.99	41.82	-6.34
Cosine-Suppression	62.83	-5.88	43.39	-11.04
Softmax-Exp	62.78	-5.43	43.01	-10.65
Ours	63.84	-5.26	43.96	-9.85

method with RPQ quantization on 10-tasks CIFAR-100 and 25-tasks TinyImageNet benchmarks. Table IV shows the impact of different diagonal similarity masks on model performance, where we compare the following configurations: Orthogonal-Only uses only the orthogonal component $\mathbf{g}_o$; Cosine-Suppression computes the similarity mask $\mathbf{M}$ as:

$$\mathbf{M}_{ij} = \begin{cases} 1 - |\cos(\mathbf{g}_p, \mathbf{u}_{pi})|, & i = j, \\ 0, & i \neq j, \end{cases} \quad (25)$$

Softmax-Exp uses a normalized exponential mask $\mathbf{M}$ computed as:

$$\mathbf{M}_{ij} = \begin{cases} \frac{\exp(-\alpha \cos^2(\mathbf{g}_p, \mathbf{u}_{pi}))}{\sum_k \exp(-\alpha \cos^2(\mathbf{g}_p, \mathbf{u}_{pk}))}, & i = j, \\ 0, & i \neq j, \end{cases} \quad (26)$$

and Ours adopts the non-normalized exponential formulation as Eq. (23). The results show that our method achieves the highest accuracy of 63.84% on 10-tasks CIFAR-100 and 43.96% on 25-tasks TinyImageNet.

The comparison with Orthogonal-Only in Table IV further clarifies the difference between SAOU and pure null-space projection. Orthogonal-Only only keeps the orthogonal component $\mathbf{g}_o$ and directly discards $\mathbf{g}_p$. This stricter constraint leads to better BWT, but it also removes useful gradient information for current task learning, resulting in lower ACC. In contrast, SAOU keeps $\mathbf{g}_o$ as the stable component and modulates $\mathbf{g}_p$ in the transition subspace with the similarity-aware mask. As shown in Table IV, SAOU improves ACC from 62.65% to 63.84% on CIFAR100 and from 41.82% to 43.96% on TinyImageNet under 4-bit quantization. This shows that SAOU is not a hard null-space projection, but preserves a modulated non-null component to improve current task learning while controlling interference.

TABLE V

HYPER-PARAMETER ANALYSIS OF θ FOR USING SAOU UPDATES IN 10-TASK CIFAR-100 AND 25-TASKS TINYIMAGENET UNDER 4-BIT QUANTIZATION.

Value of θ	CIFAR100		TinyImageNet	
	ACC (%)	BWT (%)	ACC (%)	BWT (%)
$\theta = 0.866$	63.44	-5.26	42.81	-11.08
$\theta = 0.707$	63.60	-5.66	43.30	-10.59
$\theta = 0.5$	63.84	-5.26	43.96	-9.85
$\theta = 0$	63.09	-5.78	42.17	-12.44

D. Further Analysis

Hyper-parameter Analysis of θ in SAOU. To evaluate the sensitivity of our method to the hyper-parameter θ , which controls when to apply SAOU gradient decomposition, we run experiments on 10-tasks CIFAR-100 and 25-tasks TinyImageNet under 4-bit quantization with RPQ enabled. Specifically, θ determines whether the candidate gradient $\mathbf{g}$ is aligned with the complement subspace $\bar{\mathbf{U}}_o$, using the condition $|\cos(\mathbf{g}, \bar{\mathbf{U}}_o)| > \theta$. We evaluate four values of θ : 0.866, 0.707, 0.5, and 0, corresponding to angular of 30° , 45° , 60° , and 90° , respectively.

As shown in Table V, our method demonstrates strong robustness to the choice of θ , with performance variation in ACC remaining below 1.8% across the tested range. On both benchmarks, $\theta = 0.5$ consistently yields the best balance between accuracy and forgetting, suggesting that selectively applying SAOU when the angle between the gradient and the previous subspace is within 60° enables effective knowledge retention without overly restricting plasticity. Based on these results, we set $\theta = 0.5$ for all experiments.

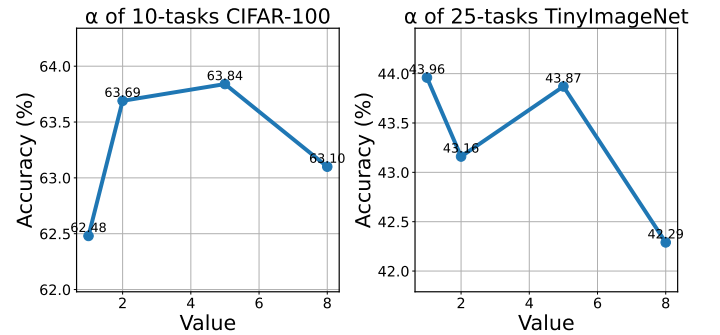


Fig. 3. Hyper-parameter analysis of α on 10-tasks of CIFAR-100 and 25-tasks of TinyImageNet under 4-bit quantization.

Hyper-parameter Analysis of α . To analyze the sensitivity of the model to hyper-parameters α , we define specific vari-

TABLE VI
ACC (%) AND BWT (%) FOR VANILLA-QCL AND OURS ON RESNET-50 AND MOBILENET-V2 UNDER 8-BIT AND 4-BIT QUANTIZATION ON CIFAR-100 AND TINYIMAGENET WITH VARYING NUMBER OF TASKS.

Backbone	Bit-width	Method	CIFAR100				TinyImageNet	
			$T = 10$		$T = 20$		$T = 25$	
			ACC (%)	BWT (%)	ACC (%)	BWT (%)	ACC (%)	BWT (%)
ResNet-50	8-bit	Vanilla-QCL	36.80	-49.98	34.19	-57.95	16.62	-57.03
		Ours	76.23	-3.66	77.80	-4.22	52.39	-15.08
	4-bit	Vanilla-QCL	32.76	-58.28	30.96	-59.24	16.36	-59.74
		Ours	75.85	-2.56	72.48	-8.08	48.98	-17.07
MobileNet-V2	8-bit	Vanilla-QCL	34.08	-45.11	27.29	-58.82	17.90	-47.70
		Ours	69.78	-3.69	72.90	-4.82	49.78	-12.89
	4-bit	Vanilla-QCL	30.02	-56.21	25.87	-62.73	17.07	-54.55
		Ours	64.69	-3.09	69.83	-5.46	45.20	-10.21

TABLE VII
EFFICIENCY ANALYSIS OF OUR METHOD ON CIFAR100 WITH RESNET-20. WE REPORT THE TRAINING-STAGE MEMORY AND COMPUTATION, TASK-BOUNDARY ADAPTATION COST, DEPLOYED MODEL SIZE, INFERENCE-SIDE EXTRA MEMORY, PER-SAMPLE INFERENCE GFLOPS, AND PER-SAMPLE INFERENCE GBOPS UNDER DIFFERENT BIT-WIDTHS.

Bit-width	Training Stage		Task Boundary		Inference Stage			
	SVD Extra Memory (MB)	Training GFLOPs (Per Sample)	SVD GFLOPs (Per Task)	RPQ Sensitivity GFLOPs (Per Sample)	Extra Memory (MB)	Model Size (MB)	Inference GFLOPs (Per Sample)	Inference GBOPs (Per Sample)
32-bit	9.104	0.042	1.144	0.126	0.000	1.156	0.042	42.991
8-bit	9.104	0.042	1.144	0.126	0.000	0.330	0.042	2.687
4-bit	9.104	0.042	1.144	0.126	0.000	0.193	0.042	0.672
2-bit	9.104	0.042	1.144	0.126	0.000	0.124	0.042	0.168

TABLE VIII
COMPARISON OF ACC (%) AND BWT (%) IN 10-TASK CIFAR-100 AND 25-TASKS TINYIMAGENET UNDER 4-BIT QUANTIZATION. OURS IS COMPARED WITH RANDOM, FULL 8-BIT, AND FULL 4-BIT SETTINGS.

Method	CIFAR100		TinyImageNet	
	ACC (%)	BWT (%)	ACC (%)	BWT (%)
Full 4bit	59.69	-6.70	39.64	-12.00
Random	62.16	-5.62	42.50	-10.33
Ours	63.84	-5.26	43.96	-9.85
Full 8bit	63.91	-5.22	44.19	-9.88

ation intervals for different hyper-parameters. Figure 3 shows the effect of hyper-parameters α on accuracy, evaluated in 10-tasks CIFAR-100 and 25-tasks TinyImageNet benchmarks under 4-bit quantization. The results lead to the following key findings: The experimental results demonstrate the robustness of our approach, with performance variations remaining below 1.67% when adjusting α within the range of $1 \rightarrow 8$. The results show that our method exhibits minimal dependence on hyper-parameter tuning, with performance variation within 2% across the tested ranges. Based on the results, we set $\alpha = 5$ for 10-tasks CIFAR-100, $\alpha = 1$ for 20-tasks CIFAR-100, and 25-tasks TinyImageNet.

Experiments under Other Backbones. To assess the generality of our approach beyond ResNet-20, we further evaluate our method on ResNet-50 and MobileNet-V2 under different quantization settings on CIFAR-100 ($T=10, 20$) and TinyImageNet ($T=25$). As shown in Table VI, our method shows consistent improvements in both ACC and BWT across all settings and backbones. These results indicate that our method is effective across different backbones, bit widths, and task sequences, maintaining a good balance between learning new tasks and retaining prior knowledge under quantization-

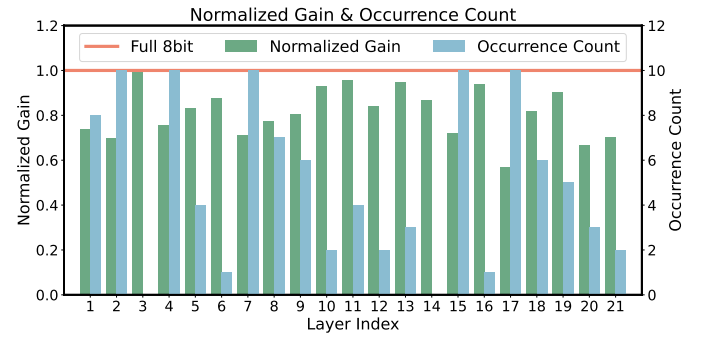


Fig. 4. Normalized gain of applying 4-bit quantization to a single layer of ResNet-20 in 10-tasks CIFAR-100, and the occurrence count of each layer exceeds the average sensitivity S_{avg} .

aware continual learning.

Analysis of Adaptive Precision Allocation and Sensitivity-Based Layer Selection. To further evaluate the effectiveness of our RPQ module, we compare the performance of our method against random precision allocation and full 8-bit quantization under 4-bit constraints in 10-tasks CIFAR-100 and 25-tasks TinyImageNet, as well as the occurrence count of each layer exceeding the average sensitivity S_{avg} in 10-tasks CIFAR-100.

As shown in Table VIII, our method achieves better performance compared to the random precision allocation, with an accuracy of 63.84% on CIFAR-100 and 43.96% on TinyImageNet, outperforming the random allocation setting which achieves 62.16% and 42.5%, respectively. The results demonstrate that, although higher precision can partially improve performance, our RPQ module differentiates layers based on sensitivity, achieving better performance. Furthermore, in

TABLE IX

ABLATION STUDY OF RPQ AND SAOU UNDER CLASS-IMBALANCED CONTINUAL LEARNING STREAMS WITH 4-BIT QUANTIZATION. THE IMBALANCE FACTOR IS SET TO 10, AND THE ORIGINAL BALANCED TEST SET IS USED FOR EVALUATION.

Idx	RPQ	SAOU	Bit-width	CIFAR100				TinyImageNet	
				T = 10		T = 20		T = 25	
				ACC (%)	BWT (%)	ACC (%)	BWT (%)	ACC (%)	BWT (%)
1	—	—	4-bit	23.12	-44.70	22.87	-51.45	15.27	-40.82
2	✓	—		24.83	-43.84	26.06	-51.00	15.84	-37.39
3	—	✓		40.07	-5.51	41.66	-14.86	26.67	-4.29
4	✓	✓		44.24	-6.96	42.92	-19.49	31.34	-5.36

Figure 4, we analyze the effectiveness of our method by showing the normalized gain and occurrence count for each layer in 10-tasks CIFAR-100. The green bars in the figure show the normalized gain of applying 4-bit quantization to a single layer, while the blue bars represent the occurrence count of each layer being assigned high bit-width across the tasks, as computed by our RPQ method in 10-tasks.

The red line represents the full 8-bit baseline. A larger gap between the green bars and the red line indicates a greater impact on performance when 4-bit quantization is applied to that layer. We observe that layers with a greater impact on performance tend to correspond with those allocated higher bit-widths, demonstrating the validity of our RPQ method. For example, layer 17, which corresponds to the residual block in block 3, has a significant impact on model performance. Meanwhile, it is consistently identified as a sensitive layer across all 10 tasks. This alignment suggests that our RPQ method is reasonable, effectively preserving critical knowledge while minimizing the performance degradation caused by low-bit quantization.

Efficiency analysis. We further analyze the efficiency of AQCL for edge deployment. As shown in Table VII, the additional cost of AQCL is limited to training-time operations, since sensitivity estimation and SVD are performed only after each task and are not used by the inference model. Specifically, RPQ only adds one extra forward-backward pass to collect the gradients for the diagonal empirical Fisher, and SAOU performs SVD on the accumulated feature covariance to update the subspaces for the following task. At inference time, the model only uses the quantized model with the learned bit-width allocation, without Fisher estimation, covariance matrices, SVD, subspace update, or gradient modulation. Therefore, AQCL introduces no extra inference memory. Meanwhile, low-bit quantization reduces the deployed model size from 1.156 MB to 0.124 MB and the inference GBOPs [71] from 42.991 to 0.168. Here, GBOPs measure bit-level operations, so they decrease when lower bit-widths are used for weights and activations. In contrast, GFLOPs are determined by the number of arithmetic operations in the model architecture, and thus remain unchanged when only the quantization precision is changed. Thus, AQCL reduces both storage and bit-level computational costs during deployment.

Class-imbalanced continual learning streams. We further evaluate our method under class-imbalanced continual learning streams. The original class-incremental task split is kept unchanged. For each task, we only subsample the training

data to form a long-tailed class distribution. For a task with K classes, the number of samples for the i -th class is set to $n_i = n_{\max} \cdot \gamma^{-i/(K-1)}$, where $i \in \{0, \dots, K-1\}$, $n_{\max}$ is the maximum class size, and $\gamma = 10$ is the imbalance factor. The original balanced test set is used for evaluation. As shown in Table IX, SAOU substantially improves both ACC and BWT under imbalanced streams. After adding RPQ, the full method achieves the highest ACC on all datasets. These results show that AQCL remains effective when the incoming data stream is class-imbalanced.

Domain-incremental learning. We also evaluate AQCL under the domain-incremental learning setting on Office-31 [72]. Here, A , D , and W denote the Amazon, DSLR, and Webcam domains in Office-31, respectively. We evaluate two domain orders, *i.e.*, $A \rightarrow D \rightarrow W$ and $W \rightarrow D \rightarrow A$, under both 4-bit and 2-bit quantization. In this setting, each task corresponds to a different domain, while the class space is shared across tasks. This setting is more challenging because the model needs to adapt to domain shifts in the incoming data stream. As shown in Table X, AQCL achieves the best ACC under both domain orders and both bit-widths. These results show that AQCL is also effective in domain-incremental learning scenarios.

Extremely low-bit regime. We further study AQCL under an extremely low-bit regime with 1-bit quantization. As shown in Table XII, AQCL improves both ACC and BWT over the Vanilla-QCL baseline on all datasets. This shows that AQCL can still alleviate forgetting and quantization damage under binary quantization. However, compared with the 2-bit and 4-bit results in the main experiments, the performance under 1-bit quantization is much lower. This suggests that AQCL cannot fully recover the severe information loss caused by 1-bit quantization. We therefore regard 1-bit continual quantization as a challenging setting and leave specialized binary training strategies as future work.

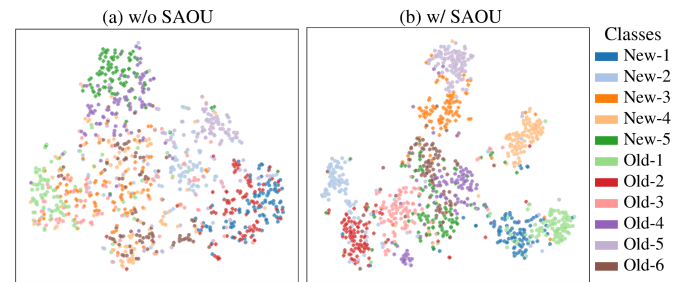


Fig. 5. t-SNE visualization on CIFAR-100 with and without SAOU.

Feature drift analysis. To analyze feature drift, we measure

TABLE X
COMPARISON OF ACC (%) AND BWT (%) UNDER THE DOMAIN-INCREMENTAL LEARNING SETTING ON OFFICE-31 WITH RESNET-50 UNDER 4-BIT AND 2-BIT QUANTIZATION.

Bit-width	Method	Office-31			
		$A \rightarrow D \rightarrow W$		$W \rightarrow D \rightarrow A$	
		ACC (%)	BWT (%)	ACC (%)	BWT (%)
4-bit	LSQ [12]	32.71	-58.80	20.51	-70.29
	LLSQ [14]	42.30	-47.27	17.95	-71.02
	IntraQ [18]	–	–	11.09	-84.38
	AlignQ [19]	46.10	-41.46	22.37	-64.91
	LifeQuant [59]	48.54	-38.96	25.65	-58.63
	Ours	57.59	-25.75	35.75	-60.68
2-bit	LSQ [12]	28.03	-65.15	15.43	-64.37
	LLSQ [14]	40.28	-49.84	15.83	-65.20
	IntraQ [18]	–	–	12.15	-73.59
	AlignQ [19]	39.06	-51.61	16.06	-64.24
	LifeQuant [59]	46.54	-42.59	17.23	-63.90
	Ours	54.30	-28.96	21.36	-78.51

TABLE XI
TASK-BY-TASK ACCURACY AND FEATURE DRIFT UNDER THE 10-TASK SETTING. WE COMPARE THE RESULTS WITH AND WITHOUT SAOU.

Metrics	Methods	Task 0	Task 1	Task 2	Task 3	Task 4	Task 5	Task 6	Task 7	Task 8	Task 9
Accuracy $\uparrow$	w/o SAOU	76.8	42.4	32.3	18.0	18.1	17.1	17.9	13.8	10.6	16.9
	w/ SAOU	76.8	68.3	71.3	67.7	70.8	64.2	61.0	64.5	62.8	66.9
	Δ	0.0	+25.9	+39.0	+49.7	+52.7	+47.1	+43.1	+50.7	+52.2	+50.0
Feature drift $\downarrow$	w/o SAOU	0.00	0.50	0.58	0.68	0.70	0.69	0.71	0.71	0.74	0.73
	w/ SAOU	0.00	0.24	0.27	0.27	0.27	0.30	0.32	0.33	0.31	0.31
	Δ	0.00	-0.26	-0.31	-0.41	-0.43	-0.39	-0.39	-0.38	-0.43	-0.42

TABLE XII
RESULTS UNDER 1-BIT QUANTIZATION. WE COMPARE AQCL WITH THE VANILLA-QCL BASELINE UNDER AN EXTREMELY LOW-BIT REGIME.

Method	CIFAR100				TinyImageNet	
	$T = 10$		$T = 20$		$T = 25$	
	ACC (%)	BWT (%)	ACC (%)	BWT (%)	ACC (%)	BWT (%)
Vanilla-QCL	20.34	-35.94	22.77	-45.84	13.85	-33.90
Ours	41.05	-4.73	38.27	-20.02	23.87	-12.35

how the representations of first task samples change after learning subsequent tasks. We compute the average ℓ_2 distance between their current features and the original features obtained after the first task. As shown in Table XI, SAOU reduces the feature drift of the first task and maintains higher task 0 accuracy. We also provide the t-SNE results in Fig. 5 as a qualitative reference. Together, these results suggest that SAOU helps alleviate representation drift in quantization-aware continual learning.

VI. CONCLUSION

In conclusion, this work introduces a quantization-aware continual learning framework to address indiscriminate bit allocation and quantization-amplified feature drift, both of which intensify catastrophic forgetting in low-bit continual learning. Our analysis shows that model performance is more sensitive to quantization in some layers than others, and that low-bit precision amplifies feature drift across tasks, highlighting the need for adaptive precision and similarity-aware updates.

Therefore, we propose Representation-Preserving Quantization (RPQ) and Similarity-Aware Orthogonal Update (SAOU), which together preserve critical knowledge by allocating more bits to important layers, suppress drift by modulating gradients within prior subspaces, and retain adaptability to new tasks. In this way, AQCL effectively relieved the coupled challenges of maintaining prior task representations and enabling efficient adaptation under strict bit-width constraints. Experiments across diverse bit-widths and task configurations show that AQCL surpasses prior methods on multi-task settings, confirming that adaptive quantization and similarity-aware updates jointly enable stable, hardware-efficient continual learning on resource-constrained edge devices. Future work will explore applying AQCL to broader architectures and modalities, as well as more realistic non-stationary scenarios on resource-limited edge devices.

REFERENCES

- [1] Y. Gao, Y. Dai, H. Li, W. Ye, J. Chen, D. Chen, D. Zhang, T. He, G. Zhang, and J. Han, "Cosurfs: 3d surface gaussian splatting with collaborative distributed learning for large-scale scene reconstruction," *International Journal of Computer Vision*, vol. 134, no. 5, p. 195, 2026.
- [2] S. Fei, X. Gao, J. Hu, X. Hou, L. Li, J. Ren *et al.*, "Knowledge distillation-based distributed dynamic 3d gaussian splatting for large scale scene reconstruction," *Expert Systems with Applications*, p. 130758, 2025.
- [3] D. Zhang, H. Li, D. He, N. Liu, L. Cheng, J. Wang, and J. Han, "Unsupervised pre-training with language-vision prompts for low-data instance segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 8642–8657, 2025.

- [4] K. Kotar and R. Mottaghi, "Interactron: Embodied adaptive object detection," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 14 860–14 869.
- [5] R. Banner, Y. Nahshan, E. Hoffer, and D. Soudry, "Acic: Analytical clipping for integer quantization of neural networks," pp. 1–16, 2018.
- [6] Y. Choukroun, E. Kravchik, F. Yang, and P. Kisilev, "Low-bit quantization of neural networks for efficient inference," in *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*. IEEE, 2019, pp. 3009–3018.
- [7] T.-A. Chen, D.-N. Yang, and M.-S. Chen, "Climbq: Class imbalanced quantization enabling robustness on efficient inferences," *Advances in Neural Information Processing Systems*, vol. 35, pp. 37 134–37 145, 2022.
- [8] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars, "A continual learning survey: Defying forgetting in classification tasks," *IEEE transactions on pattern analysis and machine intelligence*, vol. 44, no. 7, pp. 3366–3385, 2021.
- [9] H. Cha, J. Lee, and J. Shin, "Co2l: Contrastive continual learning," in *Proceedings of the IEEE/CVF International conference on computer vision*, 2021, pp. 9516–9525.
- [10] D. Zhang, Y. Li, D. Cheng, N. Wang, and J. Han, "Center-sensitive kernel optimization for efficient on-device incremental learning," *arXiv preprint arXiv:2406.08830*, pp. 1–13, 2024.
- [11] Q. Chen, C. Shui, L. Han, and M. Marchand, "On the stability-plasticity dilemma in continual meta-learning: Theory and algorithm," *Advances in Neural Information Processing Systems*, vol. 36, pp. 27 414–27 468, 2023.
- [12] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," *arXiv preprint arXiv:1902.08153*, pp. 1–12, 2019.
- [13] K. Choi, D. Hong, N. Park, Y. Kim, and J. Lee, "Qimera: Data-free quantization with synthetic boundary supporting samples," *Advances in Neural Information Processing Systems*, vol. 34, pp. 14 835–14 847, 2021.
- [14] X. Zhao, Y. Wang, X. Cai, C. Liu, and L. Zhang, "Linear symmetric quantization of neural networks for low-precision integer hardware," in *International Conference on Learning Representations*, 2020, pp. 1–16.
- [15] R. Zhao, Y. Hu, J. Dotzel, C. De Sa, and Z. Zhang, "Improving neural network quantization without retraining using outlier channel splitting," in *International conference on machine learning*. PMLR, 2019, pp. 7543–7552.
- [16] I. Hubara, Y. Nahshan, Y. Hanani, R. Banner, and D. Soudry, "Improving post training neural quantization: Layer-wise calibration and integer programming," *arXiv preprint arXiv:2006.10518*, pp. 1–15, 2020.
- [17] Y. Li, X. Dong, and W. Wang, "Additive powers-of-two quantization: An efficient non-uniform discretization for neural networks," *arXiv preprint arXiv:1909.13144*, pp. 1–15, 2019.
- [18] Y. Zhong, M. Lin, G. Nan, J. Liu, B. Zhang, Y. Tian, and R. Ji, "Intraq: Learning synthetic images with intra-class heterogeneity for zero-shot network quantization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 339–12 348.
- [19] T.-A. Chen, D.-N. Yang, and M.-S. Chen, "Alignq: Alignment quantization with admm-based correlation preservation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 12 538–12 547.
- [20] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan, "Pact: Parameterized clipping activation for quantized neural networks," *arXiv preprint arXiv:1805.06085*, pp. 1–15, 2018.
- [21] S. Jung, C. Son, S. Lee, J. Son, J.-J. Han, Y. Kwak, S. J. Hwang, and C. Choi, "Learning to quantize deep networks by optimizing quantization intervals with task loss," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 4350–4359.
- [22] J. Lee, D. Kim, and B. Ham, "Network quantization with element-wise gradient scaling," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 6448–6457.
- [23] B. Zhuang, L. Liu, M. Tan, C. Shen, and I. Reid, "Training quantized neural networks with a full-precision auxiliary module," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 1488–1497.
- [24] Q. Jin, L. Yang, and Z. Liao, "Adabits: Neural network quantization with adaptive bit-widths," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2146–2156.
- [25] J. Shen, Y. Wang, P. Xu, Y. Fu, Z. Wang, and Y. Lin, "Fractional skipping: Towards finer-grained dynamic cnn inference," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 5700–5708.
- [26] H. Yu, H. Li, H. Shi, T. S. Huang, and G. Hua, "Any-precision deep neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, 2021, pp. 10 763–10 771.
- [27] J. Yang, X. Shen, J. Xing, X. Tian, H. Li, B. Deng, J. Huang, and X.-s. Hua, "Quantization networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 7308–7316.
- [28] R. Gong, X. Liu, S. Jiang, T. Li, P. Hu, J. Lin, F. Yu, and J. Yan, "Differentiable soft quantization: Bridging full-precision and low-bit neural networks," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 4852–4861.
- [29] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, pp. 1–12, 2013.
- [30] J. Leo and J. Kalita, "Survey of continuous deep learning methods and techniques used for incremental learning," *Neurocomputing*, vol. 582, p. 127545, 2024.
- [31] Y. Lu, S. Zhang, D. Cheng, Y. Xing, N. Wang, P. Wang, and Y. Zhang, "Visual prompt tuning in null space for continual learning," *Advances in neural information processing systems*, vol. 37, pp. 7878–7901, 2024.
- [32] K. Wei, X. Yang, Z. Xu, and C. Deng, "Class-incremental unsupervised domain adaptation via pseudo-label distillation," *IEEE Transactions on Image Processing*, vol. 33, pp. 1188–1198, 2024.
- [33] K. Zhou, J. Yang, C. C. Loy, and Z. Liu, "Conditional prompt learning for vision-language models," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 16 816–16 825.
- [34] X. Xiang, Y. Tan, Q. Wan, J. Ma, A. Yuille, and G. D. Hager, "Coarse-to-fine incremental few-shot learning," in *European Conference on Computer Vision*. Springer, 2022, pp. 205–222.
- [35] H. Wang, D. Cheng, L. He, Y. Li, J. Li, N. Wang, and X. Gao, "Ekpc: Elastic knowledge preservation and compensation for class-incremental learning," *arXiv preprint arXiv:2506.12351*, pp. 1–19, 2025.
- [36] C. Wu, Q. Wu, R. Ma, K. N. Ngan, H. Li, F. Meng, and H. Qiu, "Continual cross-domain image compression via entropy prior guided knowledge distillation and scalable decoding," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 34, no. 9, pp. 8080–8092, 2024.
- [37] J. Bang, H. Kim, Y. Yoo, J.-W. Ha, and J. Choi, "Rainbow memory: Continual learning with a memory of diverse samples," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 8218–8227.
- [38] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. Torr, "Riemannian walk for incremental learning: Understanding forgetting and intransigence," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 532–547.
- [39] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "icarl: Incremental classifier and representation learning," in *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, 2017, pp. 2001–2010.
- [40] L. Bonicelli, M. Boschini, A. Porrello, C. Spampinato, and S. Calderara, "On the effectiveness of lipschitz-driven rehearsal in continual learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 31 886–31 901, 2022.
- [41] D. Yu, M. Zhang, M. Li, F. Zha, J. Zhang, L. Sun, and K. Huang, "Contrastive correlation preserving replay for online continual learning," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 34, no. 1, pp. 124–139, 2023.
- [42] Z. Ma, Y. Ma, X. Hong, H. Li, and S. Zhang, "Joint memory optimization for continual learning," *IEEE Transactions on Circuits and Systems for Video Technology*, pp. 6961–6973, 2025.
- [43] K. Li, H. Chen, J. Wan, and S. Yu, "Esdb: Expand the shrinking decision boundary via one-to-many information matching for continual learning with small memory," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 34, no. 8, pp. 7328–7343, 2024.
- [44] J. Lu and S. Sun, "Pamk: Prototype augmented multi-teacher knowledge transfer network for continual zero-shot learning," *IEEE Transactions on Image Processing*, pp. 3353–3368, 2024.
- [45] Y. Zhou, J. Yao, F. Hong, Y. Zhang, and Y. Wang, "Balanced destruction-reconstruction dynamics for memory-replay class incremental learning," *IEEE Transactions on Image Processing*, pp. 4966–4981, 2024.
- [46] Y.-S. Liang and W.-J. Li, "Inflora: Interference-free low-rank adaptation for continual learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 23 638–23 647.

- [47] J. Yu, Y. Zhuge, L. Zhang, P. Hu, D. Wang, H. Lu, and Y. He, "Boosting continual learning of vision-language models via mixture-of-experts adapters," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 23 219–23 230.
- [48] J. Serra, D. Suris, M. Miron, and A. Karatzoglou, "Overcoming catastrophic forgetting with hard attention to the task," in *International conference on machine learning*. PMLR, 2018, pp. 4548–4557.
- [49] L. He, D. Cheng, Z. Ma, H. Wang, D. Zhang, N. Wang, and X. Gao, "Ckaa: Cross-subspace knowledge alignment and aggregation for robust continual learning," *arXiv preprint arXiv:2507.09471*, pp. 1–13, 2025.
- [50] H. Wang, D. Cheng, G. Li, Z. Xu, L. He, J. Li, N. Wang, and X. Gao, "Stpr: Spatiotemporal preservation and routing for exemplar-free video class-incremental learning," *arXiv preprint arXiv:2505.13997*, pp. 1–17, 2025.
- [51] S. Dong, X. Gao, Y. He, Z. Zhou, A. C. Kot, and Y. Gong, "Ceat: Continual expansion and absorption transformer for non-exemplar class-incremental learning," *IEEE Transactions on Circuits and Systems for Video Technology*, pp. 3146–3159, 2024.
- [52] X. Wang, Z. Ji, Y. Yu, Y. Pang, and J. Han, "Model attention expansion for few-shot class-incremental learning," *IEEE Transactions on Image Processing*, pp. 4419–4431, 2024.
- [53] S. Wang, X. Li, J. Sun, and Z. Xu, "Training networks in null space of feature covariance for continual learning," in *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*, 2021, pp. 184–193.
- [54] D. Cheng, Y. Lu, L. He, S. Zhang, X. Yang, N. Wang, and X. Gao, "Mamba-cl: Optimizing selective state space model in null space for continual learning," *arXiv preprint arXiv:2411.15469*, pp. 1–18, 2024.
- [55] G. Lin, H. Chu, and H. Lai, "Towards better plasticity-stability trade-off in incremental learning: A simple linear connector," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 89–98.
- [56] Y. Hu, D. Cheng, D. Zhang, N. Wang, T. Liu, and X. Gao, "Task-aware orthogonal sparse network for exploring shared knowledge in continual learning," in *Forty-first International Conference on Machine Learning*, 2024, pp. 19 153–19 164.
- [57] D. Cheng, Y. Hu, N. Wang, D. Zhang, and X. Gao, "Achieving plasticity-stability trade-off in continual learning through adaptive orthogonal projection," *IEEE Transactions on Circuits and Systems for Video Technology*, pp. 7485–7498, 2025.
- [58] B. Qiu, H. Qiu, H. Wen, L. Wang, Y. Dai, F. Meng, Q. Wu, and H. Li, "Geodesic-aligned gradient projection for continual task learning," *IEEE Transactions on Image Processing*, pp. 1995–2007, 2025.
- [59] T.-A. Chen, D.-N. Yang, and M.-S. Chen, "Overcoming forgetting catastrophe in quantization-aware training," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 17 358–17 367.
- [60] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [61] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," pp. 1–60, 2009.
- [62] G. M. Van de Ven and A. S. Tolias, "Three scenarios for continual learning," *arXiv preprint arXiv:1904.07734*, pp. 1–18, 2019.
- [63] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the national academy of sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [64] J. Lee, H. G. Hong, D. Joo, and J. Kim, "Continual learning with extended kronecker-factored approximate curvature," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 9001–9010.
- [65] G. Saha, I. Garg, and K. Roy, "Gradient projection memory for continual learning," *arXiv preprint arXiv:2103.09762*, pp. 1–18, 2021.
- [66] S. Lin, L. Yang, D. Fan, and J. Zhang, "Trgp: Trust region gradient projection for continual learning," *arXiv preprint arXiv:2202.02931*, pp. 1–15, 2022.
- [67] Y. Kong, L. Liu, Z. Wang, and D. Tao, "Balancing stability and plasticity through advanced null space in continual learning," in *European Conference on Computer Vision*. Springer, 2022, pp. 219–236.
- [68] J. Wu, Q. Zhang, and G. Xu, "Tiny imagenet challenge," *Technical report*, pp. 1–9, 2017.
- [69] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, pp. 8024–8035, 2019.
- [70] L. Caccia, R. Aljundi, N. Asadi, T. Tuytelaars, J. Pineau, and E. Belilovsky, "New insights on reducing abrupt representation change in online continual learning," *arXiv preprint arXiv:2104.05025*, pp. 1–27, 2021.
- [71] C. Baskin, N. Liss, E. Schwartz, E. Zheltonozhskii, R. Giryas, A. M. Bronstein, and A. Mendelson, "Uniq: Uniform noise injection for non-uniform quantization of neural networks," *ACM Transactions on Computer Systems (TOCS)*, vol. 37, no. 1-4, pp. 1–15, 2021.
- [72] K. Saenko, B. Kulis, M. Fritz, and T. Darrell, "Adapting visual category models to new domains," in *European conference on computer vision*. Springer, 2010, pp. 213–226.



De Cheng received the B.S. and Ph.D. degrees from Xi'an Jiaotong University, Xi'an, China, in 2011 and 2017, respectively. He is currently an Associate Professor with the School of Telecommunications Engineering, Xidian University, Xi'an. From 2015 to 2017, he was a Visiting Scholar with Carnegie Mellon University, Pittsburgh, PA, USA. His research interests include pattern recognition, machine learning, and multimedia analysis.



Kun Gu received the B.Sc. degree from Xiangtan University, Xiangtan, China, in 2024. He is currently pursuing his M.S. degree in Information and Communication Engineering in Xidian University. His research interest is continual learning.



Lingfeng He received the B.Sc. degree from Xidian University, Xi'an, China, in 2023. He is currently pursuing his M.S. degree in Information and Communication Engineering in Xidian University. His research interests include person re-identification, unsupervised learning and continual learning.



Huaijie Wang received the B.Sc. degree from Xidian University, Xi'an, China, in 2024. He is currently pursuing his Ph.D. degree in School of Electronic Engineering in Xidian University. His research interest is continual learning.



Di Xu received the B.S. degree and Ph.D degree from Nanyang Technological University, Singapore, in 2011 and 2016, respectively. He is currently a computer vision research expert in Huawei. His research interests include 3D vision, generative AI, digital human, and virtual reality.



Nannan Wang received the B.S. degree in information and computation science from the Xi'an University of Posts and Telecommunications in 2009 and the Ph.D. degree in information and telecommunications engineering from Xidian University in 2015. From September 2011 to September 2013, he was a Visiting Ph.D. Student with the University of Technology Sydney, Australia. He is currently a Professor with the State Key Laboratory of Integrated Services Networks, Xidian University.



Xinbo Gao (M'02-SM'07) received the B.Eng., M.Sc. and Ph.D. degrees in electronic engineering, signal and information processing from Xidian University, Xi'an, China, in 1994, 1997, and 1999, respectively. From 1997 to 1998, he was a Research Fellow with the Department of Computer Science, Shizuoka University, Shizuoka, Japan. From 2000 to 2001, he was a Post-Doctoral Research Fellow with the Department of Information Engineering, the Chinese University of Hong Kong, Hong Kong. Since 2001, he has been with the School of Elec-

tronic Engineering, Xidian University. He is also a Cheung Kong Professor of the Ministry of Education of China, a Professor of Pattern Recognition and Intelligent System with Xidian University, and a Professor of Computer Science and Technology with the Chongqing University of Posts and Telecommunications, Chongqing, China. He has published 6 books and around 300 technical articles in refereed journals and proceedings. His research interests include image processing, computer vision, multimedia analysis, machine learning, and pattern recognition. Prof. Gao is also a Fellow of the Institute of Engineering and Technology and the Chinese Institute of Electronics. He has served as the general chair/cochair, the program committee chair/co-chair, or a PC member for around 30 major international conferences. He is also on the Editorial Boards of several journals, including Signal Processing (Elsevier) and Neurocomputing (Elsevier).